

# Assignment 9: Time Series Analysis

Haochuan Zhan

Spring 2026

## OVERVIEW

This exercise accompanies the lessons in Environmental Data Analytics on generalized linear models.

## Directions

1. Rename this file `<FirstLast>_A09_TimeSeries.Rmd` (replacing `<FirstLast>` with your first and last name).
2. Change “Student Name” on line 3 (above) with your name.
3. Work through the steps, **creating code and output** that fulfill each instruction.
4. Be sure to **answer the questions** in this assignment document.
5. When you have completed the assignment, **Knit** the text and code into a single PDF file.

## Set up

1. Set up your session:
  - Check your working directory
  - Load the tidyverse, zoo, and trend packages
  - Set your ggplot theme

```
getwd()
```

```
## [1] "C:/Users/zhc/Desktop/R/EDE_Spring2026/Assignments"
```

```
library(tidyverse)
```

```
## Warning: package 'tidyverse' was built under R version 4.2.3
```

```
## Warning: package 'ggplot2' was built under R version 4.2.3
```

```
## Warning: package 'tibble' was built under R version 4.2.3
```

```
## Warning: package 'tidyr' was built under R version 4.2.3
```

```
## Warning: package 'readr' was built under R version 4.2.3
```

```
## Warning: package 'purrr' was built under R version 4.2.3

## Warning: package 'dplyr' was built under R version 4.2.3

## Warning: package 'stringr' was built under R version 4.2.3

## Warning: package 'forcats' was built under R version 4.2.3

## Warning: package 'lubridate' was built under R version 4.2.3

## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
## v dplyr      1.1.4      v readr      2.1.5
## v forcats    1.0.0      v stringr   1.5.1
## v ggplot2    3.5.1      v tibble    3.2.1
## v lubridate  1.9.3      v tidyr     1.3.1
## v purrr      1.0.2
## -- Conflicts ----- tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()     masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(zoo)
```

```
## Warning: package 'zoo' was built under R version 4.2.3

##
## Attaching package: 'zoo'
##
## The following objects are masked from 'package:base':
##
##      as.Date, as.Date.numeric
```

```
library(trend)
```

```
## Warning: package 'trend' was built under R version 4.2.3
```

```
theme_set(
  theme_minimal() +
  theme(
    panel.grid.minor = element_blank(),
    plot.title = element_text(face = "bold", hjust = 0.5),
    axis.title = element_text(face = "bold")
  )
)
```

2. Import the ten datasets from the Ozone\_TimeSeries folder in the Raw data folder. These contain ozone concentrations at Garinger High School in North Carolina from 2010-2019 (the EPA air database only allows downloads for one year at a time). Import these either individually or in bulk and then combine them into a single dataframe named **GaringerOzone** of 3589 observation and 20 variables.

```
#1
files <- list.files("../Data/Raw/Ozone_TimeSeries", full.names = TRUE)

GaringerOzone <- files %>%
  map_dfr(read_csv)

dim(GaringerOzone)

## [1] 3589 20
```

## Wrangle

3. Set your date column as a date class.
4. Wrangle your dataset so that it only contains the columns Date, Daily.Max.8.hour.Ozone.Concentration, and DAILY\_AQI\_VALUE.
5. Notice there are a few days in each year that are missing ozone concentrations. We want to generate a daily dataset, so we will need to fill in any missing days with NA. Create a new data frame that contains a sequence of dates from 2010-01-01 to 2019-12-31 (hint: `as.data.frame(seq())`). Call this new data frame Days. Rename the column name in Days to "Date".
6. Use a `left_join` to combine the data frames. Specify the correct order of data frames within this function so that the final dimensions are 3652 rows and 3 columns. Call your combined data frame GaringerOzone.

```
# 3
GaringerOzone <- GaringerOzone %>%
  mutate(Date = as.Date(as.character(Date), format = "%m/%d/%Y"))

# 4
GaringerOzone <- GaringerOzone %>%
  select(Date, `Daily Max 8-hour Ozone Concentration`, DAILY_AQI_VALUE)

# 5
Days <- as.data.frame(seq(as.Date("2010-01-01"), as.Date("2019-12-31"), by = "day")) %>% rename(Date = `Date`)

# 6
GaringerOzone <- left_join(Days, GaringerOzone, by = "Date")
class
```

```
## function (x) .Primitive("class")
```

## Visualize

7. Create a line plot depicting ozone concentrations over time. In this case, we will plot actual concentrations in ppm, not AQI values. Format your axes accordingly. Add a smoothed line showing any linear trend of your data. Does your plot suggest a trend in ozone concentration over time?

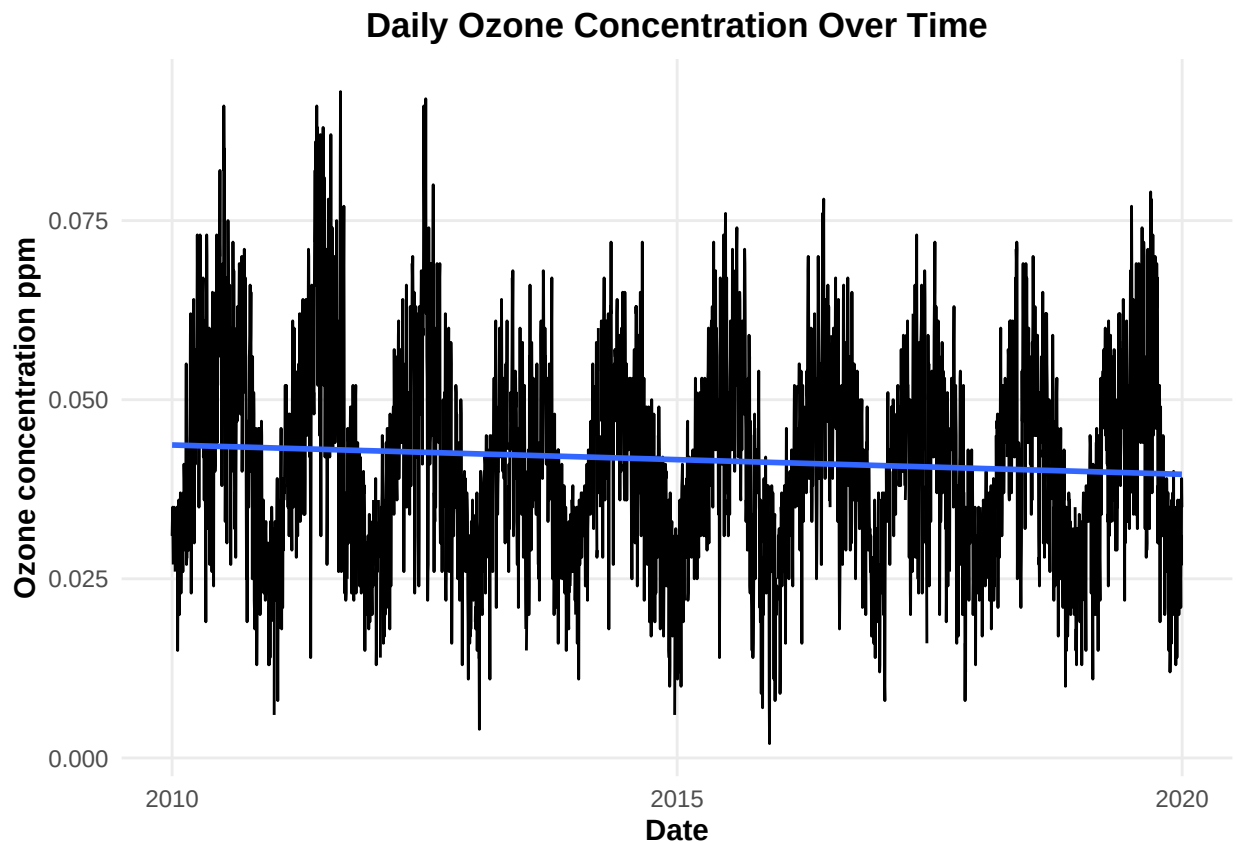
```
#7
ggplot(GaringerOzone, aes(Date, `Daily Max 8-hour Ozone Concentration`)) +
  geom_line(na.rm = TRUE) +
  geom_smooth(method = "lm", se = FALSE, na.rm = TRUE) +
  labs(
```

```

title = "Daily Ozone Concentration Over Time",
x = "Date",
y = "Ozone concentration ppm"
)

```

```
## 'geom_smooth()' using formula = 'y ~ x'
```



Answer: The plot suggests a slight downward trend in ozone concentration over time, although there is substantial short-term variation throughout the series.

## Time Series Analysis

Study question: Have ozone concentrations changed over the 2010s at this station?

8. Use a linear interpolation to fill in missing daily data for ozone concentration. Why didn't we use a piecewise constant or spline interpolation?

```

#8
GaringerOzone <- GaringerOzone %>%
  mutate(`Daily Max 8-hour Ozone Concentration` =
    na.approx(`Daily Max 8-hour Ozone Concentration`, na.rm = FALSE)) %>%
  mutate(`Daily Max 8-hour Ozone Concentration` =
    na.locf(`Daily Max 8-hour Ozone Concentration`, na.rm = FALSE)) %>%

```

```
mutate(`Daily Max 8-hour Ozone Concentration` =
  na.locf(`Daily Max 8-hour Ozone Concentration`, fromLast = TRUE, na.rm = FALSE))
```

Answer: We used linear interpolation because it gives simple estimates between nearby days. We did not use piecewise constant interpolation because it creates flat steps, and we did not use spline interpolation because it can add unrealistic curves.

9. Create a new data frame called `GaringerOzone.monthly` that contains aggregated data: mean ozone concentrations for each month. In your pipe, you will need to first add columns for year and month to form the groupings. In a separate line of code, create a new Date column with each month-year combination being set as the first day of the month (this is for graphing purposes only)

```
#9
GaringerOzone.monthly <- GaringerOzone %>%
  mutate(
    year = format(Date, "%Y"),
    month = format(Date, "%m")
  ) %>%
  group_by(year, month) %>%
  summarise(mean_ozone = mean(`Daily Max 8-hour Ozone Concentration`), .groups = "drop")

GaringerOzone.monthly <- GaringerOzone.monthly %>%
  mutate(Date = as.Date(paste(year, month, "01", sep = "-")))
```

10. Generate two time series objects. Name the first `GaringerOzone.daily.ts` and base it on the dataframe of daily observations. Name the second `GaringerOzone.monthly.ts` and base it on the monthly average ozone values. Be sure that each specifies the correct start and end dates and the frequency of the time series.

```
#10
GaringerOzone.daily.ts <- ts(
  GaringerOzone$`Daily Max 8-hour Ozone Concentration`,
  start = c(2010, 1),
  frequency = 365
)

GaringerOzone.monthly.ts <- ts(
  GaringerOzone.monthly$mean_ozone,
  start = c(2010, 1),
  end = c(2019, 12),
  frequency = 12
)
length(GaringerOzone.daily.ts)
```

```
## [1] 3652
```

```
start(GaringerOzone.daily.ts)
```

```
## [1] 2010    1
```

```
end(GaringerOzone.daily.ts)
```

```
## [1] 2020    2
```

```
frequency(GaringerOzone.daily.ts)
```

```
## [1] 365
```

```
length(GaringerOzone.monthly.ts)
```

```
## [1] 120
```

```
start(GaringerOzone.monthly.ts)
```

```
## [1] 2010    1
```

```
end(GaringerOzone.monthly.ts)
```

```
## [1] 2019   12
```

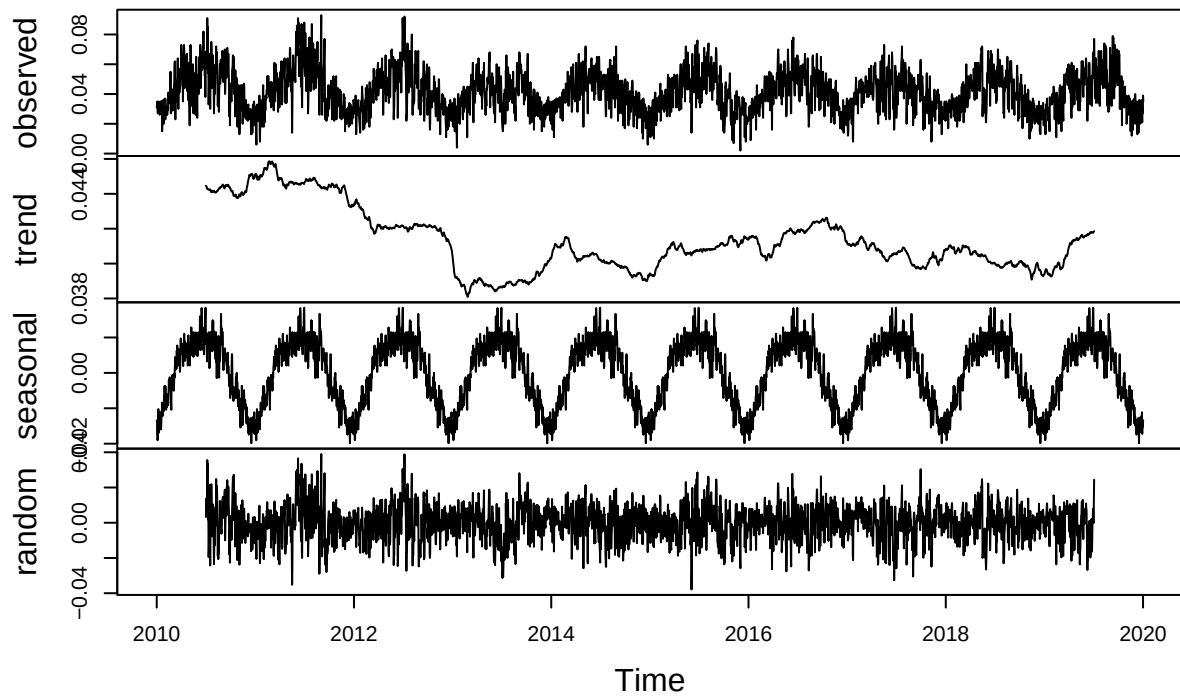
```
frequency(GaringerOzone.monthly.ts)
```

```
## [1] 12
```

11. Decompose the daily and the monthly time series objects and plot the components using the `plot()` function.

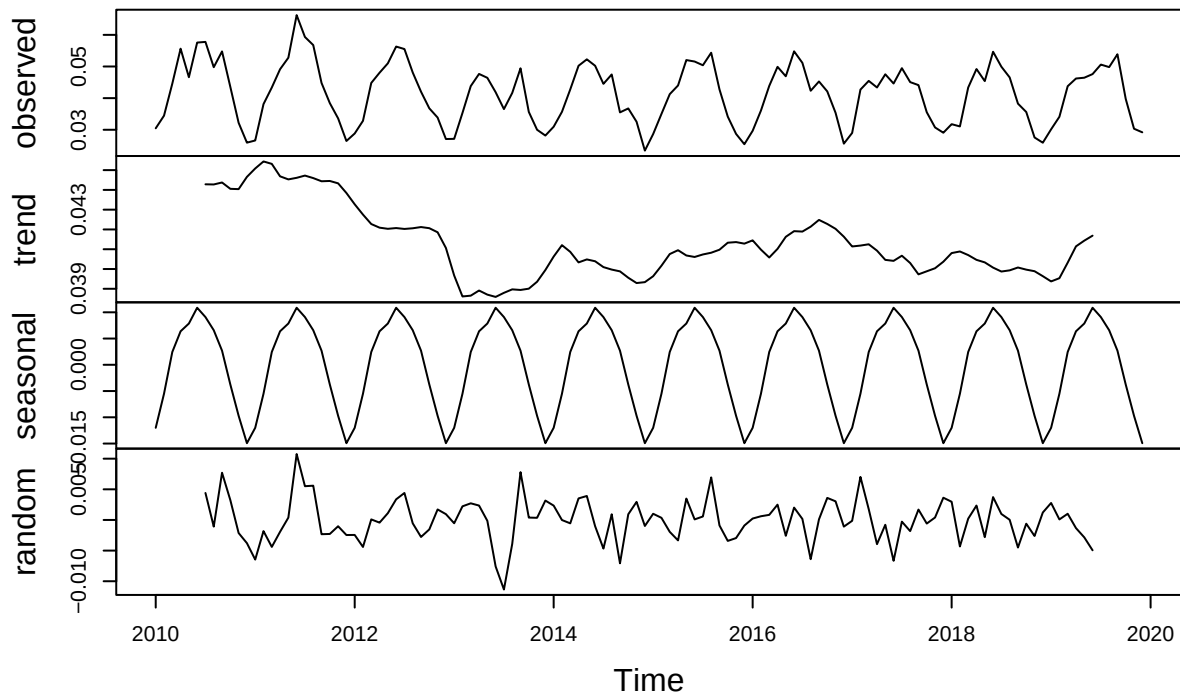
```
#11  
daily_dec <- decompose(GaringerOzone.daily.ts)  
plot(daily_dec)
```

## Decomposition of additive time series



```
monthly_dec <- decompose(GaringerOzone.monthly.ts)
plot(monthly_dec)
```

## Decomposition of additive time series



12. Run a monotonic trend analysis for the monthly Ozone series. In this case the seasonal Mann-Kendall is most appropriate; why is this?

#12

```
smk.test(GaringerOzone.monthly.ts)
```

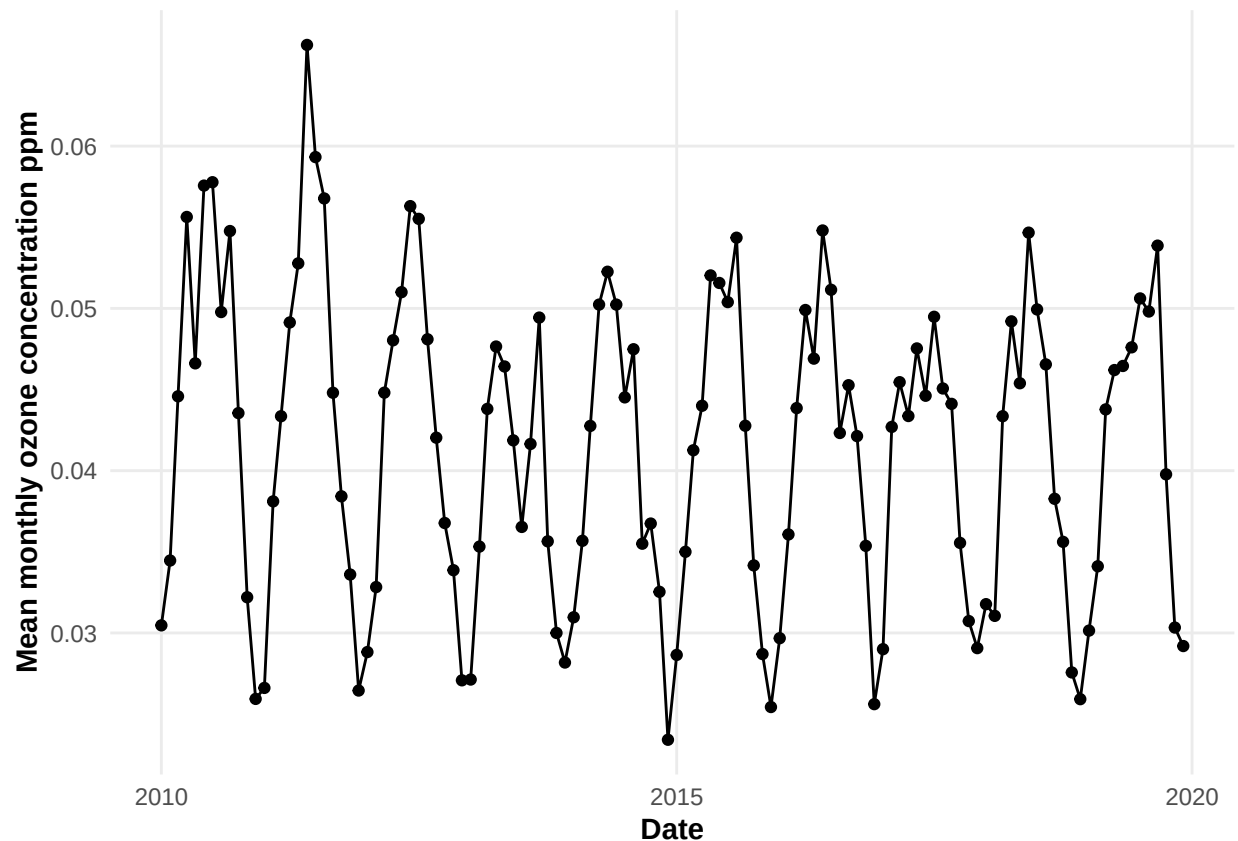
```
##
## Seasonal Mann-Kendall trend test (Hirsch-Slack test)
##
## data:  GaringerOzone.monthly.ts
## z = -1.963, p-value = 0.04965
## alternative hypothesis: true S is not equal to 0
## sample estimates:
##      S varS
## -77 1499
```

Answer: The seasonal Mann-Kendall test is best because the monthly ozone data are seasonal. The result shows a significant downward trend over time with  $z = -1.963$  and  $p = 0.04965$ .

13. Create a plot depicting mean monthly ozone concentrations over time, with both a `geom_point` and a `geom_line` layer. Edit your axis labels accordingly.

# 13

```
ggplot(GaringerOzone.monthly, aes(Date, mean_ozone)) +  
  geom_point() +  
  geom_line() +  
  labs(  
    x = "Date",  
    y = "Mean monthly ozone concentration ppm"  
  )
```



14. To accompany your graph, summarize your results in context of the research question. Include output from the statistical test in parentheses at the end of your sentence. Feel free to use multiple sentences in your interpretation.

Answer: Ozone concentrations at this station showed a slight but significant decrease over the 2010s. Even though the series had strong seasonal variation, the overall long-term trend was downward (seasonal Mann-Kendall:  $z = -1.963$ ,  $p = 0.04965$ ).

15. Subtract the seasonal component from the `GaringerOzone.monthly.ts`. Hint: Look at how we extracted the series components for the `EnoDischarge` on the lesson Rmd file.
16. Run the Mann Kendall test on the non-seasonal Ozone monthly series. Compare the results with the ones obtained with the Seasonal Mann Kendall on the complete series.

#15

```
GaringerOzone.monthly.decomp <- stl(GaringerOzone.monthly.ts, s.window = "periodic")
```

```
GaringerOzone.monthly.noseason <- GaringerOzone.monthly.ts -  
  GaringerOzone.monthly.decomp$time.series[, "seasonal"]
```

#16

```
mk.test(GaringerOzone.monthly.noseason)
```

```
##  
## Mann-Kendall trend test  
##  
## data: GaringerOzone.monthly.noseason  
## z = -2.672, n = 120, p-value = 0.00754  
## alternative hypothesis: true S is not equal to 0  
## sample estimates:  
##          S          varS          tau  
## -1.179000e+03  1.943657e+05 -1.651376e-01
```

Answer: The non-seasonal monthly ozone series still shows a significant downward trend (Mann-Kendall:  $z = -2.672$ ,  $p = 0.00754$ ). This matches the Seasonal Mann-Kendall result, but the trend is stronger after removing seasonality, since the p-value is smaller than before.